

Extended Abstract

Motivation High frequency trading (trading minute bars where the agent makes decisions every minute) can provide a nice balance between profitability and riskiness, as positions can be closed expediently when unexpected events happen. However, trading minute bars is a difficult problem for deep reinforcement learning due to the low learnable signal to noise ratio in this setting. Additionally, there is a lack of literature on applying deep reinforcement learning algorithms to high frequency trading. All these factors make it an interesting problem to solve with deep reinforcement learning.

Method Due to the lack of literature on applying deep reinforcement learning to high frequency trading, first, Proximal Policy Optimization (PPO) is used along with three different reward functions to establish baselines. The three reward functions differ in magnitudes and sparsity. All agents are trained on one stock (NVDA) from the years 2016-2019, and then tested on data from 2020-2023. After the baselines are established, two novel modifications to PPO are made. The first one, PPO- ϵ , involves adapting the clip rate parameter in PPO to make more conservative updates when the value function estimates are more uncertain. The second modification, DPPO-QR, adopts ideas from distributional reinforcement learning to learn a distribution of value functions for a given state using quantile regression (QR) to create a distributional PPO algorithm.

Implementation All environments are created using the openAI gymnasium environment, and heavily adapted to interface with the queried stock market data and action masking (for example, an agent that has purchased stock at a previous time should not be able to purchase stock until the stock has been sold, so sometimes invalid actions need to be masked out). Both PPO- ϵ and DPPO-QR involve modifications to stablebaselines3's implementation of the MaskedPPO algorithm by creating subclasses of the MaskedPPO algorithm. Especially for DPPO-QR, modifications must also be made to the replay buffer, value function updates, train step, and predict steps of the MaskedPPO algorithm class from stablebaselines3. To ensure that the environments are created properly and to ensure the baseline PPO works as implemented, the stablebaselines3's MaskedPPO is used for the PPO baselines.

Results The PPO baselines indicate that using raw returns as a reward function can cause the agent to "give up" and just not interact with the environment, while using dense and normalized reward functions can keep the agent from "giving up", but learning doesn't take place when training on 2016-2019 data for 100M iterations. By shortening the training data to only the year 2019, the dense and normalized reward functions can help PPO learn in the noisy environment in 100M training iterations. The agents trained with PPO- ϵ are more conservative (make less trades) but are less performant than PPO for the same number of training iterations. Meanwhile, DPPO-RL is able to beat PPO during training as it results in an agent collecting more rewards during training, but is unable to generalize as well to data from 2020-2023.

Discussion As expected, using dense rewards that are properly normalized provides the best chance for PPO to learn, and using less data can help reduce the time for training runs. The results for PPO- ϵ reinforce that conservative updates are being applied as compared to the PPO algorithm, and thus agents trained with PPO- ϵ need to be trained for longer than agents trained with PPO to achieve the same performance. Meanwhile, the results for DPPO-RL suggest that more data should be used for training to more accurately characterize distributions that can generalize better (train on 2016-2019 data, not just 2019 data). The results indicate that with more data and longer training runs, DPPO-RL has the potential to be more performant than PPO for high frequency trading settings.

Conclusion First, by establishing PPO baselines using three different reward functions, the dense and normalized reward function is the best when using PPO to learn agents in high frequency trading settings. By using adaptive clip rates, PPO- ϵ provides more conservative updates during training, and trains more conservative agents. Meanwhile, DPPO-RL outperforms PPO during training, but is less generalizable than PPO to data in the future. This could be mitigated by training for longer and with more data (will take around 2 weeks to train so not suitable for this project, this is future work that will be done). DPPO-RL provides promise for high frequency trading settings when the reward signal is very noisy.

Adapting Proximal Policy Optimization to High Frequency Trading

Andy Wu

Department of Computer Science
Stanford University
awu1018@stanford.edu

Abstract

Deep reinforcement learning applied to high-frequency trading is an underexplored area. Using Proximal Policy Optimization (PPO), baselines are introduced to establish the performance of PPO using three different reward functions. Then, two novel modifications to PPO are introduced, one that adaptively changes the clip rate parameter of PPO based on the uncertainty of the value function estimates, called PPO- ϵ . The second modification to PPO (DPPO-QR) involves learning the distribution of value function estimates for a given state using quantile regression. PPO- ϵ is able to train more conservative agents, but is ultimately less performant than PPO in high-frequency trading settings. Meanwhile, DPPO-QR outperforms PPO in collecting rewards during training, but suffers in generalization capabilities due to the limited data used. DPPO-QR has the potential to outperform PPO when used in high-frequency trading if longer training runs and more data are used.

1 Introduction

High frequency trading is potentially lucrative but also involves significant risk. Due to the noisy nature of financial markets and the irrationality of overall markets, it is an unsolved problem. Trading, which is a subset of financial decision making, can be casted as a Markov Decision Process (MDP) due to its sequential decision making nature. Therefore, reinforcement learning and deep reinforcement learning methods can be used to tackle this problem. In general, the frequency of trading involves trading specific bars (such as minute bars, hourly bars, or daily bars). Each bar involves price action aggregated using a consistent time discretization. For example, minute bars will aggregate information about trades that occurred in the past minute into an open price, a low price, a high price, and a close price, and agent decisions are made per minute when a new bar is created once a minute has passed.

Deep reinforcement learning has been applied to daily bars (Sarlakifar et al., 2025; Yang et al., 2020), where Sarlakifar et al. (2025) used recurrent Proximal Policy optimization (PPO) and x-LSTM networks while Yang et al. (2020) used an ensemble strategy of PPO, Advantage Actor Critic (A2C), and Deep Deterministic Policy Gradients (DDPG). While these results are promising, daily bars are significantly easier to learn since they are more low frequency, where spurious oscillations are less significant. This means that the learnable signal is large relative to the noise. However, for high frequency trading, such as trading minute bars, the learnable signal can be quite small relative to the noise (Ait-Sahalia and Yu, 2008). On the flip side, high frequency trading can be less risky than trading daily bars as there is far more time to react to unexpected situations since decisions are made every minute when the stock market is open, and positions are often closed at the end of the trading day to limit overnight risk. Therefore, it is useful to explore how to apply deep reinforcement learning algorithms effectively in these situations. Thus far, there is little literature on applying deep reinforcement learning to minute bar trading.

The contributions of this paper involve establishing baselines for minute bar trading using PPO, as well as two novel modifications to the PPO algorithm. The first modification involves adapting the clip rate to the uncertainty of the advantage estimates, while the second modification involves learning the value function distribution at a given state to account for noise systematically.

2 Related Work

The idea of learning distributions to help aid in training agents through reinforcement learning is not new. One of the first works to do so applied the idea of learning distributions to DQN (Dabney et al., 2017). Here, the authors go over a distributional bellman update, involving a target distribution and regressing the learned distribution of Q-values over states to the target distribution. When taking actions, the agent takes an action according to the action that gives the highest value in expectation. To learn the distribution, the authors introduce a loss called quantile regression (QR), which allows the neural network to learn different quantiles all at once (Dabney et al., 2017). Another method adapts this to PPO using an Implicit Quantile Network (IQN) instead of using quantile regression (Jeon et al., 2024). While the results show that the agent’s decision making process is more robust, IQN requires sampling the quantiles randomly, and the approach in Jeon et al. (2024) also requires the use of a distillation network that requires the PPO value function and the IQN output as inputs, before predicting the final value function. This requires more neural networks and possibly more stochasticity as the quantiles are sampled randomly. Another way of applying distributional reinforcement learning to actor-critic algorithms involves using quantile regression and a generative actor-critic (Tessler et al., 2019), but this approach requires the use of a delayed actor, trained by bootstrapping from a different actor, and the value function predictions are for the delayed actor. As seen, when applying distributional reinforcement learning to PPO, many additional neural networks are used in existing works. The DPPO-QR algorithm introduced in this work does not involve any other additional networks not found in the original PPO algorithm.

Another line of work centers around learning robust agents to adversarially corrupted rewards or transitions (and both corrupted rewards and transitions) (Zhang et al., 2021; Lykouris et al., 2023; Rosenberg and Mansour, 2019). While some ideas from this domain could be brought over and could potentially improve the agent, the stock trading environment is not purely adversarial, as the profit the agent obtains from closing a good position will not be reduced after the fact by what other people do in a liquid market (if the market is not liquid that is a different story, but assume all stocks traded here are liquid, as is the case on all large company stocks).

3 Method

This section goes over the methods used in the final project, which first establishes baselines using PPO, and then introduces two modifications of PPO.

3.1 Baseline PPO

First, PPO is directly applied to trading minute bars as-is. To fully evaluate PPO’s performance, three different reward functions are used. The first reward function is the pure profit/loss of the agent after it closes out the trade. Noticing that the magnitudes of this reward function are undesirable, the second reward function is $\log(\frac{P_{sell}}{P_{buy}})$, which will penalize unprofitable positions and reward profitable ones. The third reward function, similarly to the second reward function, uses the logarithmic reward function, but includes an additional reward shaping term following (Ng et al., 1999). Therefore, the first reward function is sparse with magnitudes that vary significantly, while the second reward function is also sparse but with more controlled magnitudes. Meanwhile, the third reward function has dense rewards due to reward shaping, and also has more controlled magnitudes.

3.2 Adaptive Clip Rate (PPO- ϵ)

Next, a simple modification to PPO is introduced. The clip parameter (ϵ), which controls how conservative the policy updates are, is allowed to dynamically vary based on the standard deviation of the advantages. Intuitively, if the advantage estimates have high variance, one potential reason for this is that the value function predictions are very uncertain for that trajectory. Therefore, the agent

should be updated less, and will hopefully become more conservative when trading. The algorithm for this adaptive clip rate is shown in Algorithm 1.

Algorithm 1 PPO- ϵ , this algorithm listed falls within the PPO algorithm’s batch and epoch update steps (this is done per batch, just like PPO has updates per batch and for a specific number of epochs). This algorithm only shows how to update the clip rate parameter before the regular PPO update step is applied. EMA stands for Exponential Moving Average

-
- 1: Sample batch B_t , obtain advantages A .
 - 2: $\sigma_{adv} \leftarrow stdev(A)$
 - 3: $\sigma_1 \leftarrow \alpha_1 \sigma_{adv} + (1 - \alpha_1) \sigma_1$ {EMA of standard deviation of advantages}
 - 4: $\sigma_2 \leftarrow \alpha_2 |\sigma_{adv} - \sigma_1| + (1 - \alpha_2) \sigma_2$ {EMA to get normalization scale}
 - 5: **if** $\sigma_{adv} > \sigma_1$ **then**
 - 6: $\epsilon = (\epsilon_c - 0.01) \times 2 \times (1.0 - \frac{1.0}{1.0 + e^{\frac{-2.0(\sigma_{adv} - \sigma_1)}{\sigma_2}}}) + 0.01$
 - 7: **end if**
-

As seen, when the standard deviation of the advantage is larger than the "typical" standard deviation (σ_1), the clip parameter can be decreased based on the equation in Algorithm 1. The normalization scale involves the "typical" distance (another EMA) between the standard deviation of a batch of advantages (σ_{adv}) and the EMA of σ_{adv} . The offset of 0.01 ensures that the agent is always learning, even if the flipped sigmoid function has saturated, and ϵ_c is the fixed clip rate in the original PPO algorithm.

3.3 DPPO-QR

In addition, another modification to PPO is explored. Drawing inspiration from distributional RL, the PPO value function is altered to learn a distribution across values using quantile regression. Furthermore, this approach does not involve any other neural networks or stochasticity that is found when using implicit quantile networks. This modification to PPO is the most minimal as possible while still incorporating the idea of learning a distribution of values over the state. Instead of the neural network outputting a scalar, the neural network instead outputs many values at the last layer, each representing a quantile, which is shown in Figure 1.

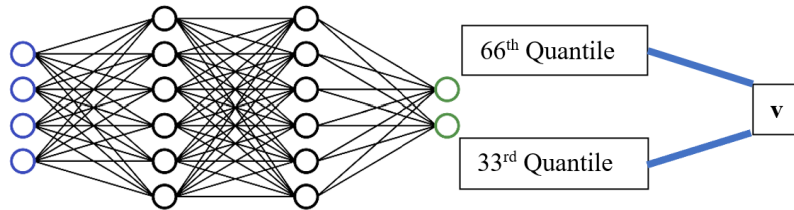


Figure 1: The value network of PPO is modified to output a distribution. In practice, the neural network outputs 100 quantiles.

To update the value function, the target value function is computed using the distributional bellman update operator and Quantile-Huber loss found in in (Dabney et al., 2017). Letting $\mathbf{v}$ denote the vector of values (each entry of the vector resulting in a specific quantile), the distributional bellman operator can be found as:

$$\mathbf{v}_{\text{target}}(s_t) = r(s_t, a_t) + \gamma * \mathbf{v}(s_{t+1}) \quad (1)$$

Letting the subscript "i" and "j" denote the specific indices of $\mathbf{v}$, one can define the difference u to be as:

$$u = \mathbf{v}_{\text{target}_i} - \mathbf{v}_j \quad (2)$$

for an arbitrary "i" and "j". Then, the huber loss can be found to be:

$$\mathcal{L}_{\text{huber}}(u) = \begin{cases} \frac{1}{2}u^2 & \text{if } |u| < \kappa \\ \kappa(|u| - \frac{1}{2}\kappa) & \text{otherwise} \end{cases} \quad (3)$$

and the quantile-huber loss can be found to be as:

$$\mathcal{L}_{value} = |\tau - \mathcal{I}(u < 0)| \times \mathcal{L}_{huber}(u) \quad (4)$$

where $\mathcal{I}$ is an indicator function and τ is the target quantile to regress towards. This loss replaces the value function loss in PPO. Furthermore, when scalar values are needed (for example, when computing advantages necessary for the policy update), one can just take the average over the vector $\mathbf{v}$ as an approximate mean. This minimal PPO implementation with distributed reinforcement learning is termed as DPPO-QR, and is shown in Algorithm 2.

Algorithm 2 DPPO-QR: Update Step. The algorithm listed falls within the PPO algorithm batch and epoch update steps (this is done per batch, and can be thought of as replacing the whole loss update step in the original PPO paper.)

- 1: Sample batch B_t
 - 2: $\mathbf{v}(s_t) \leftarrow NN(s_t)$ {Use neural network (NN) to get quantiles}
 - 3: $V \leftarrow \frac{1}{N} \sum_{i=0}^{n-1} \mathbf{v}_i$ {Approximate scalar value with a mean of all quantiles}
 - 4: $A, \mathcal{L}_{actor} \leftarrow PPO(V, RB)$ {Scalar value V used to compute advantages A using the replay buffer (RB). This will then allow us to compute the PPO actor loss, $\mathcal{L}_{actor}$. The use of clipping is the same as the original PPO loss, and the advantages are computed also using the GAE computation following normal PPO.}
 - 5: $\mathbf{v}_{target}(s_t) \leftarrow r(s_t, a_t) + \gamma * \mathbf{v}(s_{t+1})$ {Target value function distribution is obtained}
 - 6: $\mathcal{L}_{value} \leftarrow QHR(\mathbf{v}_{target}(s_t), \mathbf{v}(s_t))$ {Use the quantile huber loss to get the value function loss}
 - 7: $\mathcal{L}_{tot} = \mathcal{L}_{value} + \mathcal{L}_{actor} + \mathcal{L}_{entropy}$ {As seen, quantile huber loss replaces PPO value function loss, while the actor loss and the entropy loss follows from the original PPO paper.}
-

As seen in both Algorithms 1 and 2, these specific approaches are novel, which necessitates hyperparameter sweeps. Especially Algorithm 2, the implementation involves augmenting the replay buffer to store the distribution of values at the next states, and significant retooling of the PPO code.

4 Experimental Setup

To prevent sudden, unpredictable world events from drastically impacting an open position when the stock market is closed (can't sell if something bad happens overnight), all stock trading positions are closed before the end of the trading session. In all experiments, the hyperparameters in Table 1 are used, with variations to these "standard" hyperparameters specified if necessary. In all cases, the agent is given 9000 dollars to transact, and it has to transact 9000 dollars every single time. The agent can only decide to buy, sell, or hold a particular stock, with invalid actions masked out (for example, if the agent bought a stock previously but has yet to sell it, the agent will be unable to take the "buy" action and will only be able to "hold" or "sell" the stock).

Table 1: Hyperparameter Values

Hyperparameter	Value
Discount Factor	0.999
Env steps	64
Parallel envs	8
Batch size	512
Epochs	4
Entropy Coefficient	0.0

Since positions are closed daily, the discount factor is set to be high, at 0.999. The justification is that in the United States, the inflation is not high enough that the value of money held at the end of the day is significantly less than the value of money held at the beginning of the day. The hyperparameter "Env steps" specifies the number of steps the agent takes in the environment before updating the value function and its policy, while the number of parallel environments ("Parallel envs") results in 8

parallel agents collecting data into the replay buffer. Therefore, when updating the value function and the policy, the replay buffer will have $8 \times 64 = 512$ samples, and since the batch size is 512, all samples in the replay buffer are used (only one batch). Since the data is noisy, to not overfit to any particular trajectories, 4 epochs are run when updating the agent and value function parameters. The entropy coefficient is set to zero as a "standard" hyperparameter because higher entropy coefficients result in agents that are more stochastic but explore more. However, stochastic agents are in general, more risky and in stock trading one wants to minimize risk. Therefore, the entropy coefficient is set to zero unless otherwise specified. In this case, all agents are evaluated on data from one stock, NVDA, and the agent is trained on data from 2016-2019 before being evaluated on data from 2020-2023.

5 Results

This section goes over the results of the baselines of using PPO before going over the results of PPO- ϵ and DPPO-QR.

5.1 PPO Baselines

Using the three reward functions introduced in section 3.1, the original PPO algorithm is applied, trained on data from 2016-2019. The results shown in Figure 2 are over the whole training period. For reference, 100M training iterations takes approximately 1 day to complete.

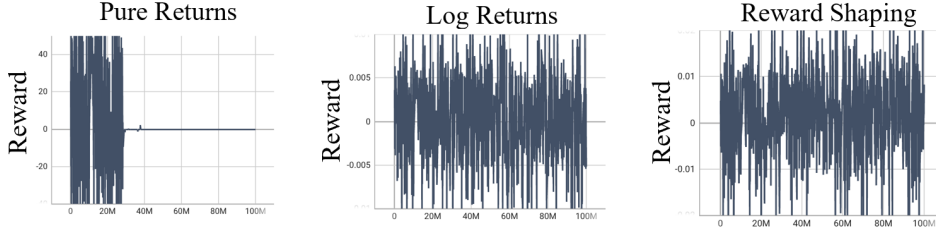


Figure 2: Reward as a function of training iterations for the three different reward functions.

As seen, when using pure returns (the first reward function), the agent decides to give up halfway due to sparse rewards and reward magnitudes varying significantly. This reward function makes it very hard for the value network in PPO to learn the correct values associated with each state. Meanwhile, the second reward function (log returns) fares better since the reward magnitude is far more bounded, but the reward signal is still sparse. The third reward function, reward shaping, also maintains an agent that is active throughout the whole training process. As seen, the learning process is extremely noisy, and all agents in this case do not have visible increases in the reward as training progresses. Despite this, the trained agents are still evaluated on data from 2020-2023, which is shown in Table 2. For reference, two different agents are also included. The "buy and hold" is an agent that just blindly buys stock on the first day of the year, holds it for one year, and sells it at the last day of the year. Meanwhile, "smoothed" is an agent trained in a setting extremely similar to the pure returns reward function setting, except that the rewards are smoothed with an exponential moving average to remove large portions of the oscillations in the stock market, and the agent is also tested in this smoothed environment. This is not a realistic setting and an agent trained in a smoothed setting but applied to the real stock market setting also does not do well.

Table 2: Agent Performance (Profit) for Experiments

Year	Pure Returns	Log Returns	Reward Shaping	Smoothed	Buy and Hold
2020	\$ 0	\$ -115	\$ -606	\$ 69,795	\$ 11230
2021	\$ 0	\$ 2131	\$ 2781	\$ 58,623	\$ 11370
2022	\$ 0	\$ 3704	\$ 1775	\$ 93,317	\$ -4589
2023	\$ 0	\$ 1633	\$ 3344	\$ 57,137	\$ 20713

As expected, the agent trained with the pure returns reward function gives zero since the agent has "given up" during training. Meanwhile, both the second and third reward function (log returns and

reward shaping) produce comparable results. All agents are not able to beat the blind "buying and holding" agent in terms of pure profit. However, it is important to note that in the years of 2020-2023, NVDA stock encountered unprecedented growth (10x in market cap), so even matching the profits of the "buy and hold" strategy would result in a very performant agent. The fact that the agent learns when rewards are smoothed indicates that the state provided to the reinforcement learning agent is sufficient to learn a good strategy. This strategy is extremely profitable, with the agent being able to increase the 9000 dollars given to it by at least a factor of 6 every year. This suggests that it is the noisy rewards in a realistic stock trading environment that are reducing the performance of the agents. PPO interacts with rewards through the value function, which implies that the PPO value function is not able to deal with noisy rewards. Therefore, more performant agents could potentially be learned by focusing on how to modify PPO to account for noisy rewards in a more systematic manner. To increase the chances of PPO learning since PPO learns better with dense rewards, from now on all reward functions use the reward shaping reward function.

5.2 PPO- ϵ

One way to account for the uncertainty in value function estimates is using PPO- ϵ , which was introduced in section 3.2. This modification for PPO forces conservative updates when the advantage estimates have higher than usual variance (potentially signaling that the value function estimates are uncertain). From Algorithm 1, two hyperparameters are introduced, α_1 and α_2 . Therefore, a hyperparameter study is done with results shown in Figure 3. In this case, to reduce the total amount of training data, which would necessitate longer training runs, the training data is reduced, where agents are now trained only on 2019 data instead of data from 2016-2019, in the hopes that more learning can be done in 100M training iterations. Agents are still evaluated from data taken from 2020-2023.

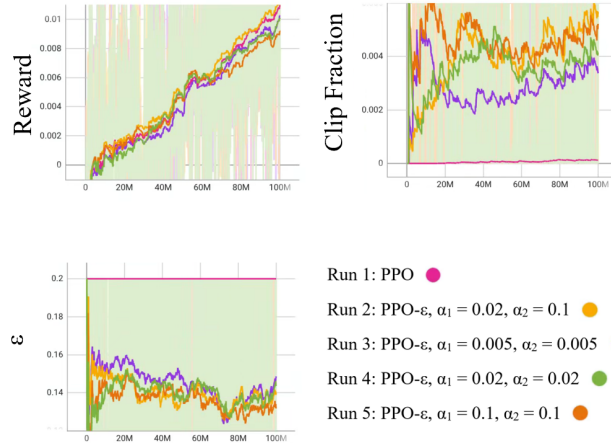


Figure 3: PPO- ϵ training runs. Runs take approximately 1 day to complete.

From Figure 3, PPO- ϵ is reducing the clipping parameter (ϵ), which is expected as some standard deviations of the advantages are exceeding the "typical" EMA smoothed standard deviation of the advantages. This is more noticeably seen in the clip fraction, where PPO almost does not clip anything, while all runs of PPO- ϵ are clipping the actor updates at a much higher rate. In this case, a lower α_1 or α_2 result in more smoothing (takes into account a longer window of the past standard deviation of the advantages to calculate the "typical" standard deviation). In this case, most agents perform relatively similarly after 100M iterations. However, reducing the data from training on 2016-2019 to just training on 2019 produces agents that learn better during training, even when using the vanilla PPO algorithm. This could be attributed to the fact that when training for 100M iterations, the total volume of 2016-2019 data is far too large and significantly longer training runs are needed to leverage the full benefit of a larger amount of data. When training for a shorter amount of time, just using 2019 results are sufficient coupled with reward shaping. After the hyperparameter study, the best performing PPO- ϵ hyperparameter combination ($\alpha_1 = 0.02$ and $\alpha_2 = 0.1$) along with vanilla

PPO is trained for 500M iterations on only the 2019 data, as the rewards seem to be still increasing after training for 100M iterations. These results are shown in Figure 4.

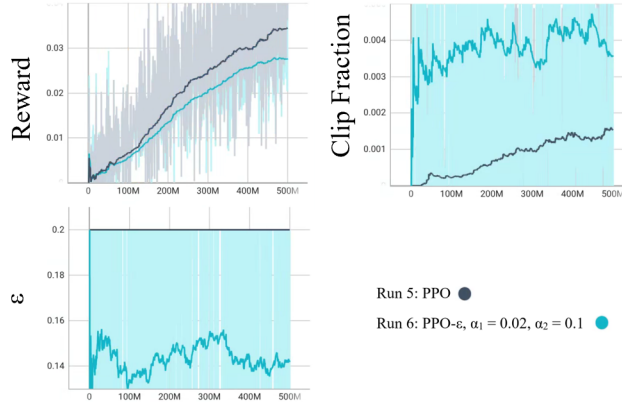


Figure 4: PPO- ϵ longer training runs. Runs take approximately 5 days to complete.

As expected, PPO- ϵ produces more conservative updates, resulting in an agent that learns slower than the original PPO algorithm. This can be seen in the rewards plot, where after 500M iterations, the rewards obtained by PPO- ϵ are lower than PPO. This is corroborated by the clip fraction, where at all given times, PPO- ϵ is clipping at least twice as much as PPO. Therefore, to obtain the same performance as PPO, it is likely that PPO- ϵ will need to be trained for longer. However, this can also be interpreted as PPO- ϵ updating less when value function estimates are more uncertain, which will hopefully produce a more conservative agent. To test this, both agents are tested on data from 2020-2023, which is shown in Table 3.

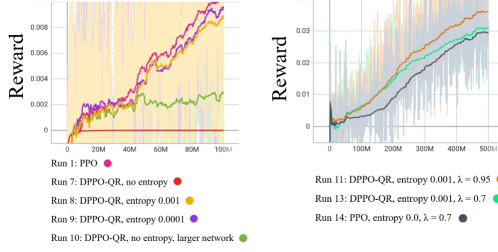
Table 3: Agent Performance (Profit) for PPO- ϵ 500M iterations

Labels	Run 5 (PPO)	Run 6 (PPO- ϵ)
2020	\$ 2080	\$ -1848
2021	\$ 3169	\$ 2901
2022	\$ 4693	\$ 3250
2023	\$ 1612	\$ 1123
Total	\$ 11554	\$ 5426
Num Trades	21880	17643

From Table 3, PPO outperforms PPO- ϵ by a factor of two when accounting for the total profit. This is expected if one observes the reward training curve, since PPO achieves a higher reward than PPO- ϵ . It is likely that training for more iterations using PPO- ϵ will help close the gap in performance. However, PPO- ϵ is successful in training an agent that is more conservative (takes less trades) as the total number of trades that the agent trained with PPO takes is over twenty percent more than the total number of trades the agent takes when trained on PPO- ϵ .

5.3 DPPO-QR

Now, a different approach is taken using distributional reinforcement learning. Similarly to above, the agent is trained on data from 2019 and tested on data from 2020-2023. Instead of using PPO- ϵ , DPPO-QR is used, as introduced in section 3.3. The number of quantiles predicted is set to 100 (value function outputs 100 values at the last layer). During the training runs, it was found that the performance is dependent on the entropy coefficient, which is a hyperparameter in PPO that encourages exploration through a more stochastic agent. Therefore, a hyperparameter study based on this parameter, as well as the λ parameter used in the generalized advantage estimate is shown in Figure 5.



(a) Training curves for hyperparameter studies.

Labels	Run 11	Run 13	Run 14
2020	\$ -164	\$ 2216	\$ -1371
2021	\$ 1426	\$ 2817	\$ 3000
2022	\$ 3101	\$ 3388	\$ 4147
2023	\$ 700	\$ 1021	\$ 4438
Total	\$ 5064	\$ 9446	\$ 10241
Num Trades	20446	20734	16056

(b) Profit/Loss for Agents.

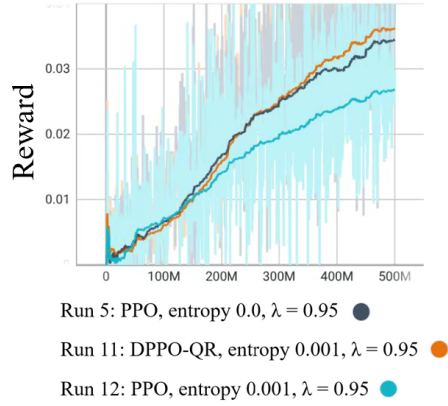
Figure 5: Training run results and corresponding agent profit/loss for DPPO-QR. The 500M iteration runs take around 5 days to complete training, while the 100M iteration runs take around 1 day to train.

As seen in Figure 5a, DPPO-QR is sensitive to the entropy coefficient. When there is no entropy, the agent sometimes fails to learn, as seen in Run 7. Even when using a larger policy and value function network, when the entropy coefficient is set to 0, the agent learns poorly relative to the rest. Intuitively, DPPO-QR requires more exploration than PPO since it is now approximating a distribution of values at various states. In order to more accurately represent the distribution of values, it must take many different, diverse actions at each state to capture a better distribution of values. Thus, a non-zero entropy coefficient is needed whereas in PPO, one could get away with using a zero entropy coefficient. Meanwhile, when using a non-zero entropy coefficient, DPPO-QR produces rewards in training that are similar to PPO. It was observed that when using a smaller value of entropy (for example using 0.0001 vs 0.001), there was significant amounts of training stochasticity (sometimes the agent would learn while other times it would not). Therefore, all future runs are carried out with an entropy coefficient of 0.001, which has far less training stochasticity.

Similarly, the λ parameter that determines the weighting applied in GAE is also varied from the suggested value of 0.95 to something lower to evaluate the effect of this parameter (PPO suggests using a value of 0.95). The training curves are shown for runs 11, 13 and 14. When looking at the training curves, it is seen that DPPO-QR using $\lambda = 0.95$ outperforms both DPPO-QR and PPO using $\lambda = 0.7$. This suggests that DPPO-QR has the ability to outperform PPO. However, when evaluating on data from 2020-2023, a different story emerges. Interestingly, PPO does the best (highest profit while taking the smallest number of trades), while DPPO-QR with $\lambda = 0.7$ falls behind very slightly (total profit of \$9446 vs \$10241). Meanwhile, DPPO-QR with $\lambda = 0.95$ does the worst, only achieving half the profit of PPO. This is counter-intuitive since when training, DPPO-QR with $\lambda = 0.95$ achieves the highest reward during training, which could suggest that DPPO-QR requires more data to accurately capture the distributions associated with different states, and it has potentially slightly overfit to only the 2019 data. Training on data from 2016-2019, and training for potentially 2000M runs instead of 500M runs may help this issue significantly. In that case, DPPO-QR is likely to outperform PPO due to less overfitting. This run would take around 2 weeks to complete and thus is left for future work.

For a final comparison, the best performing PPO and DPPO-QR runs are compared. For DPPO-QR, $\lambda = 0.95$ with an entropy coefficient of 0.001 is used, while for PPO, $\lambda = 0.95$ and an entropy coefficient of 0.0 is used. For reference, PPO with $\lambda = 0.95$ with an entropy coefficient set to 0.001 is also shown to highlight that using zero entropy trains more performant agents when PPO is used. These results are shown in Figure 6.

From Figure 6, the optimal entropy coefficient is indeed 0.0 for PPO. Figure 6a shows that DPPO-QR can outperform even the best performing PPO, showing promise that using a distributional approach can help handle noisy rewards. However, as seen in Figure 6b, DPPO-QR is unable to generalize as well when tested on 2020-2023 data, as the agent is less performant than PPO (only generated half the profits as compared to the best performing PPO agent). As stated above, DPPO-QR likely needs more data to learn a more accurate, generalizable distribution of values over different states and should be trained on all the data available (2016-2019) instead of just data from 2019. Since DPPO-QR has the potential to be more performant than PPO during training, if it learns distributions of values over states that can generalize better, it could be more performant than PPO when tested on data from 2020-2023.



(a) Training curves for final comparison

Labels	Run 5	Run 11	Run 12
2020	\$ 2080	\$ -164	\$ 449
2021	\$ 3169	\$ 1426	\$ 2736
2022	\$ 4693	\$ 3101	\$ 1016
2023	\$ 1612	\$ 700	\$ 2802
Total	\$ 11555	\$ 5064	\$ 7005
Num Trades	21880	20446	21751

(b) Profit/Loss for Agents.

Figure 6: Final comparisons for DPPO-QR. The 500M iteration runs take around 5 days to complete training.

6 Discussion

The introduced DPPO-QR could potentially also be extended to other settings where rewards are noisy, not just for high frequency trading. Throughout this project, one of the difficulties faced was the length of the experiments, as oftentimes longer training runs would take around 5 days to complete. While some runs could be run in parallel, hyperparameter studies would still take around a full week to run at times. A limitation of this work is that DPPO-QR suffers from generalization and likely needs more data to train than PPO as discussed in the previous sections.

7 Conclusion

PPO applied to high-frequency trading of minute bars is discussed, where baselines are first established, followed by two modifications to the PPO algorithm involving an adaptive clip rate parameter (PPO- ϵ) and a distributional PPO approach using quantile regression (DPPO-QR). PPO- ϵ learns agents that are more conservative than PPO, while DPPO-QR is able to outperform PPO in training settings but will require more data to generalize better so that it can outperform PPO in future settings. Overall, DPPO-QR provides a promising approach that could potentially outperform PPO in noisy reward settings, and experimentation will continue in the future, mainly on quantifying the effect of distributional RL and launching longer training runs with more data to help DPPO-RL generalize better.

8 Team Contributions

- **Paper Author:** Did everything

Changes from Proposal There has not been many changes to the proposal. This project follows what has been outlined, with minor changes to only focus on adapting PPO to high frequency trading.

References

- Yacine Ait-Sahalia and Jialin Yu. 2008. *High frequency market microstructure noise estimates and liquidity measures*. Technical Report. National Bureau of Economic Research.
- Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. 2017. Distributional Reinforcement Learning with Quantile Regression. arXiv:1710.10044 [cs.AI] <https://arxiv.org/abs/1710.10044>

- WooJae Jeon, KangJun Lee, and Jeewoo Lee. 2024. PG-Rainbow: Using Distributional Reinforcement Learning in Policy Gradient Methods. arXiv:2407.13146 [cs.LG] <https://arxiv.org/abs/2407.13146>
- Thodoris Lykouris, Max Simchowitz, Aleksandrs Slivkins, and Wen Sun. 2023. Corruption-robust exploration in episodic reinforcement learning. arXiv:1911.08689 [cs.LG] <https://arxiv.org/abs/1911.08689>
- Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 278–287.
- Aviv Rosenberg and Yishay Mansour. 2019. Online Convex Optimization in Adversarial Markov Decision Processes. arXiv:1905.07773 [cs.LG] <https://arxiv.org/abs/1905.07773>
- Faezeh Sarlakifar, Mohammadreza Mohammadzadeh Asl, Sajjad Rezvani Khaledi, and Armin Salimi-Badr. 2025. A Deep Reinforcement Learning Approach to Automated Stock Trading, using xLSTM Networks. arXiv:2503.09655 [cs.CE] <https://arxiv.org/abs/2503.09655>
- Chen Tessler, Guy Tennenholtz, and Shie Mannor. 2019. Distributional policy optimization: An alternative approach for continuous control. *Advances in Neural Information Processing Systems* 32 (2019).
- Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. 2020. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the first ACM international conference on AI in finance*. 1–8.
- Xuezhou Zhang, Yiding Chen, Xiaojin Zhu, and Wen Sun. 2021. Robust Policy Gradient against Strong Data Corruption. arXiv:2102.05800 [cs.LG] <https://arxiv.org/abs/2102.05800>